NeST Network Simulation

Comprehensive Architecture & Code Analysis Guide

Foundation Concepts in NeST

The provided Python scripts utilize NeST (Network Simulation Tools) to emulate real-world networking scenarios. Under the hood, NeST leverages Linux kernel features to create highly accurate network models.

Core Infrastructure Mechanisms

- **Namespaces:** Commands like `nest.Router()` and `nest.Node()` create isolated Linux network namespaces. Each behaves as a standalone machine with its own routing table and interfaces.
- **veth Pairs:** The `nest.Connect()` function establishes a Virtual Ethernet pair—essentially a virtual cable plugging two namespaces together.
- **NetEm (Network Emulator):** Methods like `.set_attributes(bandwidth="...", delay="...")` interface with the Linux kernel's Traffic Control (tc) subsystem to apply delays, packet loss, and rate limits.

Scenario 1: Campus Network with Internet & Departments

This scenario models a university campus with isolated department LANs (CSE, ECE, ME) routing through a core network to an ISP.

Architectural Details

- **Topology Design:** End hosts connect to L2 switches, which then uplink to routing gateways. R1 acts as a Core Router, R2 as a Gateway, R3 as an Internet Edge, and R4 as the ISP.
- **Subnetting Strategy:** Each department receives a standard `/24` subnet (e.g., `192.168.10.0/24` for CSE). The router-to-router links utilize `/24` links from the `10.0.0.0` block.

> **Code Implementation Focus: Static Routing**
>
> Because dynamic routing protocols (like OSPF/BGP) are not configured, static routing is manually established using `.add_route("Destination", "Next_Hop")`. For example, R1 must explicitly be told that to reach the ME department (`192.168.30.0/24`), it must forward packets to R2 (`10.0.0.2`). Default routes (`0.0.0.0/0`) push unknown traffic, like internet requests, upwards to the ISP.

Scenario 2: Multi-Site Corporate Network

This script emulates an enterprise with a Head Office (Bengaluru) and branches (Pune, Hyderabad) connected over constrained WAN links.

Architectural Details

- **Hub-and-Spoke Topology:** The ISP router serves as the central hub, terminating links from all three enterprise sites.
- **WAN Constraints:** Physical distance and leased-line limitations are simulated by setting lower bandwidths (e.g., `5mbit`) and higher latency (`30ms`) on the branch connections.

> **Code Implementation Focus: Router-on-a-Stick (Inter-VLAN)**
>
> At the Head Office, multiple departments (HR, Engineering) share one router interface. The code executes raw Linux commands (`ip link add link eth0 name eth0.10 type vlan id 10`) to create 802.1Q tagged sub-interfaces. This allows a single physical cable to carry traffic for multiple isolated VLANs, routing them centrally.

Scenario 3: Hospital Network with QoS

This critical infrastructure scenario connects a hospital to a remote clinic over a severely restricted 512 Kbps WAN link, requiring traffic prioritization.

Architectural Details

- **Intentional Bottleneck:** The WAN link's `512kbit` limit guarantees congestion when administrative file transfers and critical medical data compete.

- **VLAN Isolation:** Doctor PCs and Administrative PCs are placed in separate VLANs (`.20` and `.30`) to allow packet tagging based on origin.

> **Code Implementation Focus: Linux Traffic Control (QoS)**
>
> The code uses Hierarchical Token Bucket (HTB) queuing via the `tc` command. It establishes a parent class capped at 512 Kbps. It then creates a high-priority child class (`1:10`) guaranteeing 300 Kbps. A packet filter (`tc filter ... match ip src 192.168.20.0/24`) ensures that any data originating from the Doctor's subnet is forced into this priority queue, bypassing congestion.

Scenario 4: E-Commerce Company Network

A multi-branch e-commerce environment featuring a dedicated Data Center (hosting Web and DB servers) and secure Internet access for internal users.

Architectural Details

- **Data Center Centralization:** Servers are grouped in the `192.168.40.0/24` subnet directly off the Mumbai Core Router, mimicking an on-premise data center.
- **External Transit:** The Mumbai router manages all branch transit and serves as the single egress point to the ISP.

> **Code Implementation Focus: Network Address Translation (NAT)**
>
> Internal IPs are non-routable on the public internet. The script executes `iptables -t nat -A POSTROUTING -o eth3 -j MASQUERADE` on the Mumbai router. This modifies outgoing packets, translating private source IP addresses to the router's public-facing interface IP, thereby enabling internet access for internal nodes.

Scenario 5: Smart City Network

A high-capacity, centralized municipal network connecting Public Services, CCTV Surveillance, and Public Wi-Fi through a main City Hall gateway.

Architectural Details

- **High Throughput Links:** Unlike previous WAN scenarios, Smart City links utilize high capacities (e.g., `50mbit` for Wi-Fi, `20mbit` for Services) to support simultaneous UDP (CCTV) and TCP (Database) flows.
- **Star Topology:** City Hall acts as the central star node routing all inter-department traffic.

> **Code Implementation Focus: IP Forwarding**
>
> To ensure the City Hall Linux namespace acts as a true router, the script explicitly injects `sysctl -w net.ipv4.ip_forward=1`. This kernel flag permits the node to receive packets on one interface (e.g., from CCTV) and forward them out another interface (e.g., to the Database), forming the backbone of the city network.

Scenario 6: University Network with Hostel

A modern enterprise split-architecture design, cleanly separating internal Core routing from external Border routing tasks.

Architectural Details

- **Core vs. Border Separation:** `R-CORE` aggregates all internal VLANs (Admin, Academic, Hostel) over high-speed links (`100mbit`). `R-BORDER` solely handles the slower ISP link (`50mbit`) and security boundary.

> **Code Implementation Focus: Route Summarization & Edge NAT**
>
> Instead of the Border router needing a route for every individual department, it uses a summary route: `.add_route("192.168.0.0/16", "10.0.0.1")`. This efficiently tells the Border that any packet destined for the entire `192.168.x.x` space belongs inside the university. NAT is applied exclusively on the Border router, abstracting internal complexities from the ISP.